Experiment 10

AIM: Build Student Management REST API using ASP.NET Core (Middleware, API, Project Structure)

Code:

Models\

Student.cs:

```
using System.ComponentModel.DataAnnotations;

namespace StudentManagementAPI.Models;

public class Student
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Student Name is required.")]
    [StringLength(80, MinimumLength = 2)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    [StringLength(50)]
    public string Department { get; set; } = string.Empty;

    [Range(1, 8)]
    public int Semester { get; set; }

    [Range(0.0, 10.0)]
    public double Cgpa { get; set; }
```

```
    public DateTime EnrollmentDate { get; set; } = DateTime.Now;
}
```

Middleware\

RequestLoggingMiddleware.cs:

```
using System.Diagnostics;

namespace StudentManagementAPI.Middleware;

public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(RequestDelegate next, ILogger<RequestLoggingMiddleware> log
    ger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();
        var method = context.Request.Method;
        var path = context.Request.Path;

        _logger.LogInformation("[HTTP INCOMING] Method: {Method} | Path: {Path} | Time: {Time}"
    ,
            method, path, DateTime.Now.ToString("HH:mm:ss.fff"));

        await _next(context);

        stopwatch.Stop();
```

```
        var statusCode = context.Response.StatusCode;

        _logger.LogInformation("[HTTP COMPLETED] Method: {Method} | Path: {Path} | Status: {Sta
    tus} | Elapsed: {Elapsed}ms",
            method, path, statusCode, stopwatch.ElapsedMilliseconds);
    }
}
```

Controllers\

StudentsController.cs:

```
using Microsoft.AspNetCore.Mvc;
using StudentManagementAPI.Models;

namespace StudentManagementAPI.Controllers;

[ApiController]
[Route("api/[controller]")]
[Produces("application/json")]
public class StudentsController : ControllerBase
{
    private static readonly List<Student> Students =
    [
        new Student
        {
            Id = 1,
            Name = "Anubhav Goraguddi",
            Email = "anubhav.goraguddi@marwadiuniversity.ac.in",
            Department = "Computer Science and Engineering",
            Semester = 5,
            Cgpa = 9.45,
            EnrollmentDate = new DateTime(2024, 8, 1)
        },
        new Student
```

```
    {
        Id = 2,
        Name = "Rohan Sharma",
        Email = "rohan.sharma@marwadiuniversity.ac.in",
        Department = "Information Technology",
        Semester = 5,
        Cgpa = 8.80,
        EnrollmentDate = new DateTime(2024, 8, 1)
    },
    new Student
    {
        Id = 3,
        Name = "Priya Varma",
        Email = "priya.varma@marwadiuniversity.ac.in",
        Department = "Computer Science and Engineering",
        Semester = 5,
        Cgpa = 9.15,
        EnrollmentDate = new DateTime(2024, 8, 1)
    }
];

// GET: api/Students
[HttpGet]
[ProducesResponseType(typeof(IEnumerable<Student>), StatusCodes.Status200OK)]
public ActionResult<IEnumerable<Student>> GetStudents()
{
    return Ok(Students);
}

// GET: api/Students/5
[HttpGet("{id}")]
[ProducesResponseType(typeof(Student), StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
```

```
public ActionResult<Student> GetStudent(int id)
{
    var student = Students.FirstOrDefault(s => s.Id == id);
    if (student == null)
    {
        return NotFound(new { message = $"Student with ID {id} not found." });
    }
    return Ok(student);
}

// POST: api/Students
[HttpPost]
[ProducesResponseType(typeof(Student), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public ActionResult<Student> CreateStudent([FromBody] Student student)
{
    if (!ModelState.IsValid) return BadRequest(ModelState);

    student.Id = Students.Count == 0 ? 1 : Students.Max(s => s.Id) + 1;
    student.EnrollmentDate = DateTime.Now;
    Students.Add(student);

    return CreatedAtAction(nameof(GetStudent), new { id = student.Id }, student);
}

// PUT: api/Students/5
[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public IActionResult UpdateStudent(int id, [FromBody] Student updatedStudent)
{
    if (id != updatedStudent.Id) return BadRequest(new { message = "ID mismatch." });
```

```
        var existing = Students.FirstOrDefault(s => s.Id == id);
        if (existing == null) return NotFound(new { message = $"Student #{id} not found." });

        existing.Name = updatedStudent.Name;
        existing.Email = updatedStudent.Email;
        existing.Department = updatedStudent.Department;
        existing.Semester = updatedStudent.Semester;
        existing.Cgpa = updatedStudent.Cgpa;

        return NoContent();
    }

    // DELETE: api/Students/5
    [HttpDelete("{id}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public IActionResult DeleteStudent(int id)
    {
        var student = Students.FirstOrDefault(s => s.Id == id);
        if (student == null) return NotFound(new { message = $"Student #{id} not found." });

        Students.Remove(student);
        return NoContent();
    }
}
```

Program.cs:

```
using StudentManagementAPI.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI(c =>
{
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "Student REST API v1");
    c.RoutePrefix = string.Empty;
});

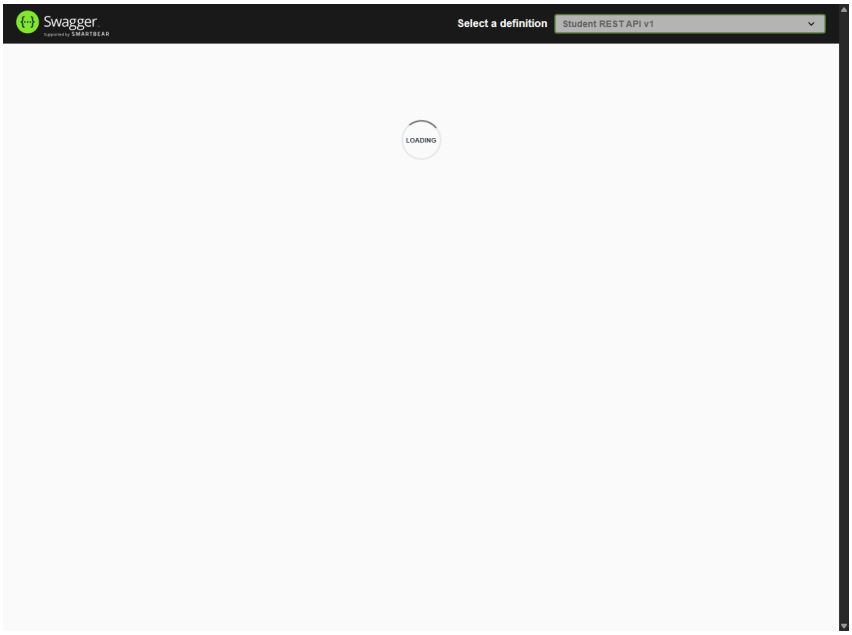
// Register Custom Request Logging Middleware
app.UseMiddleware<RequestLoggingMiddleware>();

app.UseAuthorization();
app.MapControllers();

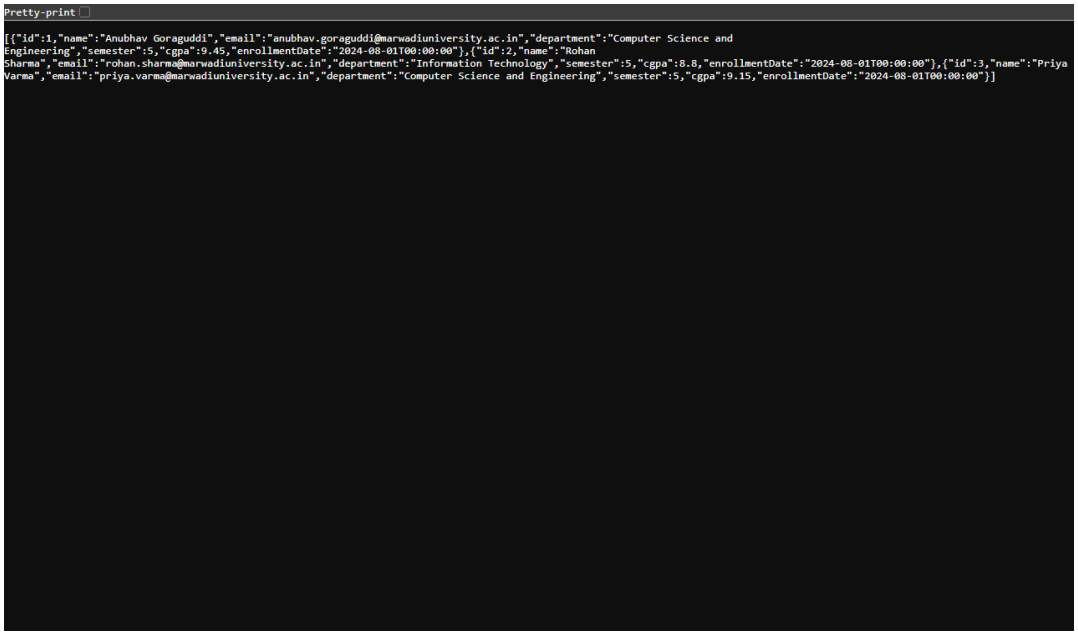
app.Run();
```

Output: Screenshot:

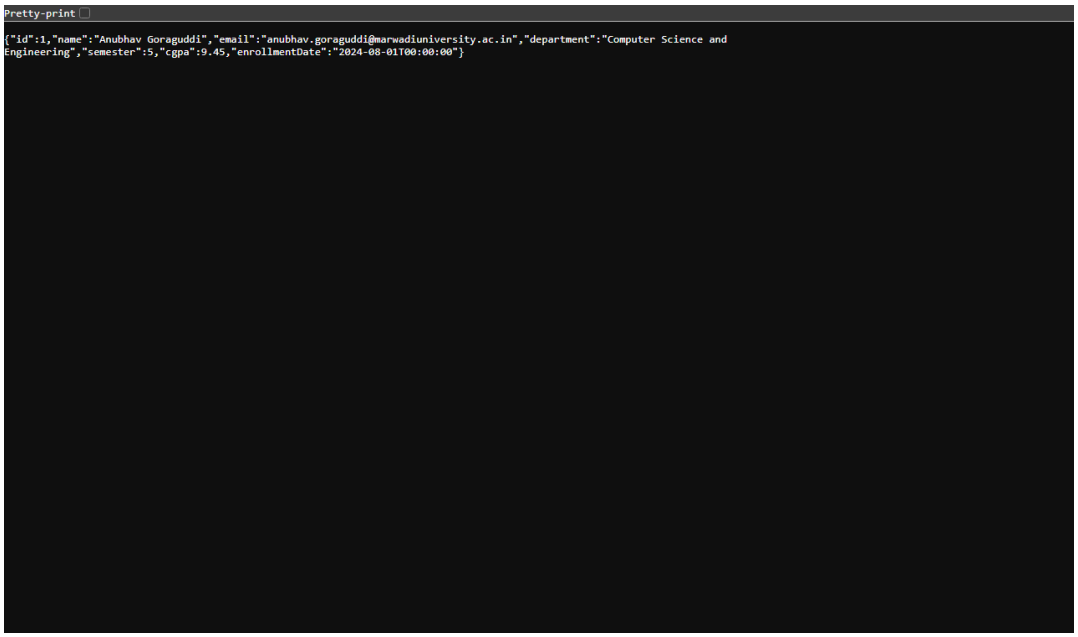
https://localhost:5001/swagger (OpenAPI / Swagger UI Documentation)



https://localhost:5001/api/Students (GET All Students - JSON Response)



https://localhost:5001/api/Students/1 (GET Student by ID - JSON Response)



GitHub URL: https://github.com/Anubhavg1123/Practical-10